

RakshaSafe

AI-Powered Women Safety and Disaster Emergency Response
System

Viva Preparation Pack

Student: Vivek Lorikprasad Sahani | Roll No. 42

Guide: Asst. Prof. Sourabh Suryawanshi

D.T.S.S. College of Commerce (Autonomous), Mumbai - 400097

Academic Year 2026-27

01 — Project Overview

One-page summary you should be able to say in 60–90 seconds.

Basic Details

Field	Value
Project title	RakshaSafe — AI-Powered Women Safety and Disaster Emergency Response System
Student	Vivek Lorikprasad Sahani
Roll No.	42
Guide	Asst. Prof. Sourabh Suryawanshi
College	D.T.S.S. College of Commerce (Autonomous), Mumbai – 400097
Academic Year	2026–27
Testing date on record	02-Oct-2026

What the project is (plain words)

RakshaSafe is a web-based emergency response platform. A person in danger (women safety issue, accident, fire, flood, etc.) can raise an **incident** with one tap, attach their GPS location, and the system will:

1. **Assess the risk** of that incident automatically (Low / Medium / High / Critical).
2. **Show nearby emergency facilities** (hospitals, police, fire stations) on a map.
3. **Notify** the user's emergency contacts (in-app now; SMS/Email are recorded as *not configured* rather than faking delivery).
4. Let an **admin** track, verify and assign the incident to a **rescue team**.
5. Let a **responder** (field team member) see only their team's assignments and move them through the rescue workflow.

Three services, one system

Layer	Technology	Port	Responsibility
Frontend	React 19 + TypeScript + Vite + Tailwind CSS 4 + Leaflet	5173	UI, map, SOS flow, admin dashboard
Backend (API)	Node.js + Express + TypeScript + Mongoose (MongoDB)	5000	Auth, incidents, risk orchestration, notifications, admin
AI service	Python 3.12 + FastAPI	8000	Deterministic risk scoring engine (<i>raksha-risk-v1</i>)

The backend calls the AI service for scoring, and always has a **local fallback** if the AI

service is down — the system never stops working just because AI is unavailable.

The one-line "AI" story (be honest)

The risk engine is **rule-based / deterministic**, not a black-box ML model. It is named **raksha-risk-v1**. It uses documented weights (incident priority, disaster flag, verified reports, unverified reports, active nearby incidents) and returns a score 0–100 plus a level. Google Gemini is an **optional explanation layer only** — it writes a human-friendly sentence about the score, but it **never** changes the score and is never required. This is a strength, not a weakness: results are reproducible and explainable.

Main modules

- **Authentication & Roles** — register, login, JWT, safe user profile. Roles: **USER**, **ADMIN**, **RESPONDER**.
- **Incident Management** — create/read/update/cancel incidents with status workflow **REPORTED → ACKNOWLEDGED → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED (+ CANCELLED)**.
- **Location** — GPS capture (never fabricated; honest denied/timeout/unavailable states), reverse geocoding, address search.
- **Risk Assessment** — score + level + contributing factors, stored per incident.
- **Nearby Resources** — DB facilities + live OpenStreetMap (Overpass) discovery; OSM results are never stored.
- **Unsafe Area Reports** — crowd-sourced unsafe-location reports with verification.
- **Emergency Contacts** — per-user contacts, exactly one primary.
- **Notifications** — in-app events recorded truthfully; SMS/Email honestly report **NOT_CONFIGURED**.
- **Rescue Teams & Assignment** — team membership, assignment workflow.
- **Admin Panel** — live MongoDB aggregation dashboard, user/team/facility/report mgmt, audit log.
- **Responder Panel** — own-team assignments, incident detail, status updates.
- **Enrichment** — weather (Open-Meteo) and map context around a location.

Database (15 Mongoose collections)

users, locations, incidents, incidentupdates, riskassessments, unsafeareareports, emergencycontacts, facilities, rescueteams, rescueassignments, notifications, adminlogs/adminlogs, disastercategories, reports, reportcounters, riskzones (15 domains).

Verified test evidence (from Chapter 6 — use these exact numbers)

- **Unit testing:** 36 suites, 127 checks, all passed.
- **Integration testing:** 67 cases passed.
- **API functional (live):** 101 cases across 27 modules, all passed.
- **Browser / E2E:** 16 journeys, 15 passed; T14 found layout defect **D1** (honest).
- Testing is done at **5 levels**.

Important: these are the only test numbers you should quote. Do not invent extra numbers.

Honest notes / known mismatches (say these confidently if asked)

6. **README.md** in the repo is **stale** — it still says "no business modules implemented yet", but the project is fully implemented. Do not read the README as the source of truth.
7. The original scope baseline mentioned **USER** and **ADMIN**; the implementation adds a third role **RESPONDER**. This is documented as a deliberate extension for field teams.
8. Frontend linting uses **oxlint** (not ESLint). Tests use the ****Node.js** built-in test runner (**`node:test`**) with **`tsx`** — not Jest. **Integration tests do use Supertest**** for HTTP-level assertions and **mongodb-memory-server** for an in-memory database.
9. SMS/Email delivery is **not configured** (no third-party provider keys). The system is honest about this instead of pretending messages were sent.

Deployment

- Frontend is built for static hosting (Cloudflare Pages style) and can also be packaged as an Android app via **Capacitor** (**appId: com.rakshasafe.app**).
- The repo does **not** commit deployment secrets or provider keys; env vars are supplied at deploy time.

Elevator pitch (say this)

"RakshaSafe is a three-tier emergency response system. A woman or any citizen raises an SOS incident from the React app. The Express backend stores it in MongoDB and calls a FastAPI risk engine that scores the danger from 0 to 100. Admins see a live dashboard and assign rescue teams; responders update the field status. We use OpenStreetMap for nearby hospitals and police, Open-Meteo for weather, and JWT-based role security. The risk model is fully deterministic and explainable, with an optional Gemini layer that only explains the score — it never changes it."

02 — Easy English Speech Answers

Short, simple, ready-to-speak answers. Read once before the viva, then say them in your own voice. Keep each answer to 3–6 sentences.

1. What is your project?

RakshaSafe is an AI-powered women safety and disaster emergency response system. It is a web application where a person can raise an emergency incident, share their live location, and get help. The system automatically calculates how serious the incident is, shows nearby hospitals and police stations, and alerts the user's emergency contacts. Admins and rescue teams can then track and respond to the incident.

2. Why did you choose this topic?

Women safety and disaster response are real problems in India. During an emergency, people lose time searching for helpline numbers or explaining their location. We wanted one system that captures the incident, the location, and the risk level in a single action, and connects the citizen to responders quickly.

3. What is the main AI part?

The AI part is a risk assessment engine called `raksha-risk-v1`. It takes the incident priority, whether it is a disaster, how many verified and unverified reports are nearby, and how many active incidents are in the area. It combines these using fixed weights and produces a risk score from 0 to 100 and a level — Low, Medium, High, or Critical. It is rule-based, so the results are explainable and repeatable.

4. Is it machine learning?

No, it is a deterministic rule-based engine, not a trained ML model. We chose this on purpose because in an emergency the result must be transparent and reproducible. We can show exactly why a score came out the way it did. This makes it safer and easier to defend than a black box.

5. What is the role of Gemini in your project?

Gemini is optional. It only writes a friendly explanation sentence about the score, like "this

incident is critical because it is a flood with several verified reports nearby." It never calculates the score and never writes to the database. If Gemini is not available, the system uses its own built-in explanation and keeps working normally.

6. What is your technology stack?

The frontend is React 19 with TypeScript, Vite, and Tailwind CSS, and we use Leaflet for maps. The backend is Node.js with Express and TypeScript, and we use Mongoose to talk to MongoDB. The AI service is Python with FastAPI. So it is a three-tier architecture: frontend, backend, and AI service.

7. Why did you use three separate services instead of one?

Separation of concerns. The UI team can change the frontend without touching the backend.

The risk engine is written in Python because it is a data and logic service, and it can be scaled or updated independently. It also means if the AI service is down, the main application still works using its fallback logic.

8. What database did you use and why?

MongoDB with Mongoose. Emergency data is flexible — some incidents have a location, some have photos or extra fields, and reports have different shapes. MongoDB's document model handles this naturally. We defined 15 collections, each with a Mongoose schema for validation.

9. How does a user raise an incident?

The user opens the SOS page. There is a simple four-step flow: fill the incident details, capture the location, review everything, and submit. The app asks the browser for GPS permission; if the user denies it, we honestly show "location denied" and let them search an address manually instead. We never make up a location.

10. What happens after the incident is submitted?

The backend saves the incident in MongoDB, records the location, and calls the risk engine to get a score. It records notification events — an in-app notification for the user and notification records for each opted emergency contact. Then the incident appears on the

user's dashboard and in the admin dashboard for tracking.

11. How does the risk scoring work?

Each priority level has a base score: Low is 15, Medium is 35, High is 60, and Critical is

10. If it is a disaster, we add 5. Each verified nearby report adds 6, up to a maximum of 30.

Each unverified report adds 2, up to 10. Active incidents nearby add 3 each, up to 15. The

total is clamped between 0 and 100. Below 25 is Low, below 50 is Medium, below 75 is High, and

75 or above is Critical.

12. What is the incident status workflow?

An incident starts as REPORTED. It then moves to ACKNOWLEDGED, then ASSIGNED to a rescue team, then IN_PROGRESS, then RESOLVED, and finally CLOSED. At any point it can be CANCELLED.

Every status change is recorded with a timestamp and who made the change.

13. What are the user roles?

There are three roles. USER is a normal citizen who raises incidents. ADMIN manages users, teams, facilities, reports, and assignments, and sees the dashboard. RESPONDER is a field team member who sees only their own team's assignments and updates the rescue status.

14. How is security handled?

Passwords are hashed with bcrypt using 12 salt rounds, and they are never stored or returned in plain text. After login we issue a JSON Web Token that contains only the user id and role.

Protected routes verify the token and re-check the user in the database. Admin and responder routes also check the role and the database connection.

15. How do you stop abuse of the API?

We have a lightweight in-memory rate limiter. For example, creating incidents is limited to 10 requests per minute per IP, and unsafe reports have the same limit. There is also a

`requireDb` middleware that returns a clean 503 error instead of letting requests hang when MongoDB is down.

16. What external services do you use?

OpenStreetMap Nominatim for address search and reverse geocoding, Overpass API for finding nearby hospitals and police stations, Open-Meteo for weather (no API key needed), and optionally Google Gemini for explanations. OSM discovery results are never stored in our database.

17. How do you handle failures of external services?

Gracefully. If weather is unavailable, the app shows "Weather unavailable" and the incident still works. If the OSM lookup fails, the nearby route returns a controlled 502. If the AI service is unreachable, the backend uses its own fallback risk calculation. Nothing external is allowed to break the core SOS flow.

18. What is the admin dashboard?

It is a live dashboard built from MongoDB aggregation queries. It shows user counts, incident counts by status, priority, and type, recent incidents, rescue team status, assignment status, facility status, report verification, risk level distribution, notification statuses, and recent admin activity. Every number comes from live data — nothing is hard-coded.

19. What is the responder panel?

Responders are rescue team members. Their panel shows only assignments for teams they belong to, and the incident details for those assignments. They can move an assignment through states like ASSIGNED, EN_ROUTE, ON_SCENE, and COMPLETED. This is scoped by the token, so a responder can never see another team's data.

20. What is the unsafe area report feature?

It lets users report unsafe locations, for example a poorly lit street or a dangerous crossing. Reports have a category and severity and can be verified by admins. Verified reports increase the risk score of nearby incidents, so community input directly improves safety.

21. How did you test the project?

At five levels: unit testing, integration testing, API functional testing, end-to-end browser testing, and manual testing. The unit tests are 36 suites with 127 checks, all passing. The

integration tests are 67 cases. The live API functional tests are 101 cases across 27 modules, all passing. The browser end-to-end tests cover 16 journeys; 15 passed and one found a layout defect that we recorded.

22. What defect did you find in testing?

In journey T14, the browser test found a layout defect labelled D1. We are honest about it in the test evidence. It is a display issue, not a functional failure, and it is documented rather than hidden.

23. Why is testing important in your project?

Because it is a safety system. If the risk score is wrong or an incident is lost, someone could be in danger. Testing proves that the core flows — register, login, raise incident, assess risk, notify, assign — actually work under both normal and failure conditions.

24. What are the limitations of your project?

SMS and Email delivery are not configured because there is no paid provider. The AI is rule-based rather than a trained model. It is designed mainly for a single-instance deployment because the rate limiter is in memory. And the map data depends on OpenStreetMap's availability.

25. What is your future scope?

We can integrate real SMS and Email providers, add real-time WebSocket updates, train a machine learning model once we have enough real incident data, add offline support for low network areas, add more Indian languages, and add wearables or a shake-to-SOS feature. We can also add a mobile app using our existing Capacitor setup.

26. Why React and TypeScript?

React gives us reusable components and fast UI updates, which matters during an emergency. TypeScript catches type errors at compile time, so there are fewer runtime bugs. Together they give a component-based, type-safe frontend.

27. Why Express for the backend?

Express is lightweight, widely used, and easy to structure with routes, controllers, models,

middleware, and services. It lets us keep clean separation between the HTTP layer and the business logic, and it works well with the Node.js ecosystem.

28. Why FastAPI for the AI service?

FastAPI is fast, has automatic request validation with Pydantic, and is built for Python data services. Python is the natural language for scoring logic and for calling AI libraries like Gemini later. It also gives us automatic interactive API documentation.

29. What did you learn from this project?

We learned full-stack development, how to design a REST API, how to model data in MongoDB, how to write a secure authentication system, how to integrate third-party APIs, and how to build an explainable scoring engine. We also learned to test systems honestly, including testing failure cases like a down database or a down AI service.

30. What is unique about your project?

Three things. First, an explainable deterministic risk engine instead of a black-box model. Second, honest engineering — the app never fakes a location or a sent message. Third, a complete three-tier system with real role separation between users, admins, and responders, integrated with live map, weather, and geocoding services.

03 — Hinglish Explanation

Hinglish = Hindi bolne mein aasan, English mein technical words. Ye answers bolne ke liye hain, likhne ke liye nahi. Apni natural awaaz mein bolo.

1. Project kya hai?

RakshaSafe ek AI-powered women safety aur disaster emergency response system hai. Ye ek web application hai jahan koi bhi vyakti ek emergency incident raise kar sakta hai, apni live location share kar sakta hai, aur help mil sakti hai. System automatically decide karta hai ki incident kitna serious hai, paas ke hospitals aur police stations dikhata hai, aur user ke emergency contacts ko alert karta hai. Uske baad admin aur rescue team incident ko track karke respond karte hain.

2. Ye topic kyun choose kiya?

Women safety aur disaster response India mein real problem hai. Emergency ke time log helpline number dhoondhne mein ya apni location samjhane mein time gawaate hain. Hum ek aisa system banana chahte the jo ek hi action mein incident, location aur risk level capture kare, aur citizen ko responder se jode.

3. AI ka hissa kya hai?

AI ka main hissa ek risk assessment engine hai jiska naam **raksha-risk-v1** hai. Ye incident ki priority, disaster hai ya nahi, paas mein kitni verified aur unverified reports hain, aur kitne active incidents hain — ye sab leke fixed weights se combine karta hai. Result mein 0 se 100 tak ka risk score aur ek level aata hai — Low, Medium, High ya Critical. Kyunki ye rule-based hai, result explainable aur repeatable hota hai.

4. Kya ye machine learning hai?

Nahi, ye deterministic rule-based engine hai, trained ML model nahi. Humne ye jaan-bujhkar choose kiya kyunki emergency mein result transparent aur reproducible hona chahiye. Hum exactly bata sakte hain ki score kyun aaya. Ye black box se safer aur defend karne mein aasan hai.

5. Gemini ka role kya hai?

Gemini optional hai. Ye sirf score ke baare mein ek friendly explanation sentence likhta hai, jaise "ye incident critical hai kyunki ye flood hai aur paas mein kai verified reports hain."

Ye kabhi score calculate nahi karta aur kabhi database mein nahi likhta. Agar Gemini available nahi hai, system apna built-in explanation use karta hai aur normal चलता rehta hai.

6. Technology stack kya hai?

Frontend: React 19, TypeScript, Vite, Tailwind CSS, aur maps ke liye Leaflet. Backend: Node.js with Express and TypeScript, aur MongoDB ke liye Mongoose. AI service: Python with FastAPI. Matlab ye three-tier architecture hai — frontend, backend aur AI service.

7. Teen alag services kyun, ek kyun nahi?

Separation of concerns. Frontend change karne se backend pe effect nahi padta. Risk engine Python mein hai kyunki ye data aur logic service hai, aur ise alag se scale ya update kar sakte hain. Aur agar AI service down ho jaye, main application apne fallback logic se चलता rehta hai.

8. Database kaunsa use kiya aur kyun?

MongoDB with Mongoose. Emergency data flexible hota hai — kuch incidents mein location hoti hai, kuch mein extra fields, reports ka shape alag hota hai. MongoDB ka document model ye naturally handle karta hai. Humne 15 collections banayi, har ek ke liye Mongoose schema se validation.

9. User incident kaise raise karta hai?

User SOS page kholta hai. Wahan simple four-step flow hai: incident details bharo, location capture karo, review karo, aur submit karo. App browser se GPS permission maangta hai; agar user deny kar de, to hum honestly "location denied" dikhate hain aur address manually search karne dete hain. Hum location kabhi bana kar nahi boltay.

10. Incident submit hone ke baad kya hota hai?

Backend incident ko MongoDB mein save karta hai, location record karta hai, aur risk engine ko call karke score laata hai. Notification events record hote hain — user ke liye in-app

notification aur har opted emergency contact ke liye notification record. Phir incident user ke dashboard aur admin dashboard mein tracking ke liye dikhta hai.

11. Risk scoring kaise hoti hai?

Har priority ka base score hai: Low 15, Medium 35, High 60, Critical 80. Agar disaster hai to 5 add karte hain. Har verified nearby report 6 add karti hai, max 30 tak. Har unverified report 2 add karti hai, max 10 tak. Active nearby incidents har ek 3 add karte hain, max 15 tak. Total ko 0 se 100 ke beech clamp karte hain. 25 se kam Low, 50 se kam Medium, 75 se kam High, aur 75 ya usse zyada Critical.

12. Incident ka status workflow kya hai?

Incident REPORTED se start hota hai. Phir ACKNOWLEDGED, phir ASSIGNED (rescue team ko), phir IN_PROGRESS, phir RESOLVED, aur finally CLOSED. Kabhi bhi CANCELLED ho sakta hai. Har status change timestamp aur kisne kiya — ye sab record hota hai.

13. User roles kaunse hain?

Teen roles hain. USER normal citizen hai jo incidents raise karta hai. ADMIN users, teams, facilities, reports aur assignments manage karta hai aur dashboard dekhta hai. RESPONDER field team member hai jo sirf apni team ke assignments dekhta hai aur rescue status update karta hai.

14. Security kaise handle ki?

Passwords bcrypt se hash hote hain, 12 salt rounds ke saath, aur plain text mein kabhi store ya return nahi hote. Login ke baad hum JSON Web Token dete hain jisme sirf user id aur role hota hai. Protected routes token verify karte hain aur user ko database se dobara check karte hain. Admin aur responder routes role aur database connection bhi check karte hain.

15. API ka misuse kaise rokthe ho?

Hamare paas ek lightweight in-memory rate limiter hai. Jaise incident create karna 10 requests per minute per IP tak limited hai, unsafe reports bhi wahi. Ek `requireDb` middleware bhi hai jo MongoDB down hone par clean 503 error deta hai, request ko hang nahi hone deta.

16. External services kaunsi use ki?

OpenStreetMap Nominatim address search aur reverse geocoding ke liye, Overpass API paas ke hospitals aur police stations dhoondhne ke liye, Open-Meteo weather ke liye (API key nahi chahiye), aur optional Google Gemini explanation ke liye. OSM ke discovery results hamare database mein kabhi store nahi hote.

17. External service fail ho jaye to?

Gracefully handle karte hain. Weather unavailable ho to app "Weather unavailable" dikhata hai aur incident phir bhi kaam karta hai. OSM lookup fail ho to nearby route controlled 502 return karta hai. AI service na mile to backend apna fallback risk calculation use karta hai. Koi bhi external cheez core SOS flow ko tod nahi sakti.

18. Admin dashboard kya hai?

Ye ek live dashboard hai jo MongoDB aggregation queries se banta hai. User counts, incident counts status/priority/type ke hisaab se, recent incidents, rescue team status, assignment status, facility status, report verification, risk level distribution, notification statuses, aur recent admin activity dikhata hai. Har number live data se aata hai — kuch hard-coded nahi hai.

19. Responder panel kya hai?

Responder rescue team ka member hota hai. Uska panel sirf uski team ke assignments aur un incidents ki details dikhata hai. Wo assignment ko ASSIGNED, EN_ROUTE, ON_SCENE, COMPLETED jaise states mein move kar sakta hai. Ye token se scoped hai, isliye responder kabhi doosri team ka data nahi dekh sakta.

20. Unsafe area report feature kya hai?

Isse users unsafe locations report kar sakte hain, jaise kam roshni wali gali ya dangerous crossing. Reports mein category aur severity hoti hai aur admin unhe verify kar sakta hai. Verified reports paas ke incidents ka risk score badha deti hain, matlab community input directly safety improve karta hai.

21. Testing kaise ki?

Paanch levels par: unit testing, integration testing, API functional testing, end-to-end browser testing, aur manual testing. Unit tests 36 suites, 127 checks, sab pass. Integration 67 cases. Live API functional 101 cases, 27 modules, sab pass. Browser E2E 16 journeys; 15 pass hue aur ek ne ek layout defect pakda jo humne record kiya.

22. Testing mein kya defect mila?

Journey T14 mein browser test ne ek layout defect pakda, jiska label D1 hai. Hum test evidence mein iske baare mein honest hain. Ye display issue hai, functionality ka nahi, aur humne ise hide nahi kiya, document kiya.

23. Testing itni important kyun?

Kyunki ye safety system hai. Agar risk score galat ho ya incident lost ho jaye to koi khatre mein aa sakta hai. Testing prove karti hai ki core flows — register, login, incident raise, risk assess, notify, assign — normal aur failure dono conditions mein kaam karte hain.

24. Project ki limitations kya hain?

SMS aur Email delivery configured nahi hai kyunki paid provider nahi hai. AI rule-based hai, trained model nahi. Rate limiter memory mein hai, isliye ye single-instance deployment ke liye hai. Aur map data OpenStreetMap ki availability par depend karta hai.

25. Future scope kya hai?

Real SMS aur Email providers integrate kar sakte hain, real-time WebSocket updates add kar sakte hain, enough real data aane par machine learning model train kar sakte hain, low network areas ke liye offline support add kar sakte hain, aur zyada Indian languages add kar sakte hain. Existing Capacitor setup se mobile app bhi bana sakte hain.

26. React aur TypeScript kyun?

React reusable components aur fast UI updates deta hai, jo emergency mein zaroori hai. TypeScript compile time par type errors pakadta hai, isliye runtime bugs kam hote hain. Dono milkar component-based, type-safe frontend dete hain.

27. Backend ke liye Express kyun?

Express lightweight hai, widely used hai, aur routes, controllers, models, middleware aur services ke saath structure karna easy hai. Ye HTTP layer aur business logic ko clean alag rakhne deta hai, aur Node.js ecosystem ke saath acche se chalta hai.

28. AI service ke liye FastAPI kyun?

FastAPI fast hai, Pydantic se automatic request validation deta hai, aur Python data services ke liye bana hai. Python scoring logic aur Gemini jaise AI libraries ke liye natural language hai. Ye automatic interactive API documentation bhi deta hai.

29. Is project se kya seekha?

Humne full-stack development seekha, REST API design, MongoDB data modelling, secure authentication, third-party API integration, aur ek explainable scoring engine banana. Humne honest testing bhi seekhi, including failure cases jaise database down ya AI service down.

30. Aapke project mein unique kya hai?

Teen cheezein. Pehla, black-box model ki jagah explainable deterministic risk engine. Doosra, honest engineering — app kabhi fake location ya fake sent message nahi dikhata. Teesra, ek complete three-tier system jismein users, admins aur responders ke beech real role separation hai, live map, weather aur geocoding ke saath.

Viva ke liye kuch useful lines (bolne ke liye)

- "Sir, ye rule-based engine hai, isliye result reproduce ho sakta hai — same input, same score."
- "Humne deliberately location kabhi fabricate nahi ki. Agar user permission deny kare to hum honestly 'denied' state dikhate hain."
- "Agar AI service down ho jaye to system apne fallback se chal^र rehta hai. Core SOS flow kabhi rukta nahi."
- "Passwords bcrypt se hash hote hain, plain text mein kahin store nahi hote."

- "Test evidence Chapter 6 mein hai — unit 36 suites / 127 checks, integration 67 cases, API 101 cases / 27 modules, E2E 16 journeys."

04 — Viva Questions & Answers (100+)

Grouped A–J. Answers are short and exam-ready. Where a number is quoted, it is a real project number. Never invent statistics beyond these.

A. Project & Domain

A1. What is your project?

RakshaSafe — an AI-powered women safety and disaster emergency response system. Citizens raise incidents with location; the system scores risk, shows nearby resources, notifies contacts, and lets admins/responders manage the response.

A2. Who is the intended user?

Three users: a citizen (USER) who reports emergencies, an administrator (ADMIN) who manages the system, and a field responder (RESPONDER) who handles assignments.

A3. What problem does it solve?

It removes delay in emergencies by capturing incident + location + risk in one action and connecting the citizen to responders and nearby help.

A4. What types of incidents are supported?

Two broad types: SAFETY and DISASTER. Categories include Women Safety, Medical Emergency, Accident, Fire, Flood, Earthquake, and Other Emergency.

A5. What is an "incident"?

A single emergency report created by a user, containing type, category, description, priority, status, and an optional location.

A6. What are the four risk levels?

LOW, MEDIUM, HIGH, CRITICAL.

A7. What is the incident lifecycle?

REPORTED → ACKNOWLEDGED → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED, plus CANCELLED.

A8. Is this a real deployed product?

It is a working academic project with real integrations (live maps, weather, geocoding). It is production-shaped but not commercially deployed.

A9. What is your individual contribution?

Full-stack implementation of the three services, the risk engine, the security layer, and the test suite, under the guidance of Asst. Prof. Sourabh Suryawanshi.

A10. Why women safety *and* disaster response together?

Both share the same core need — fast, location-aware emergency reporting and coordinated response. One engine serves both, which is more efficient and more useful.

B. Architecture & Design

B1. Describe your architecture.

Three tiers: React frontend (5173) → Express REST API (5000) → FastAPI AI service (8000), with MongoDB as the data store. The backend orchestrates everything and owns persistence.

B2. Why three services?

Separation of concerns, independent scaling, and fault isolation. If the AI service fails, the backend still serves using a local fallback.

B3. What design pattern does the backend use?

Layered MVC-style: routes → controllers → services → models, with shared middleware.

B4. How does the frontend talk to the backend?

Through a small typed `api()` helper. Base URL is `VITE_API_BASE_URL` or `/api`, and Vite proxies `/api` to port 5000 in development.

B5. How does routing work in the frontend?

Hash-based routing (`#/dashboard`, `#/sos`, `#/incident/:id`). No React Router dependency; a small custom hash-route module handles navigation.

B6. What does the backend do if MongoDB is down?

It still starts. The `requireDb` middleware rejects DB-dependent routes with a clean 503,

while enrichment routes (weather, nearby, geocode) keep working.

B7. What is the role of `server.ts` vs `app.ts`?

`app.ts` builds the Express app (middleware, routes, error handling). `server.ts` binds it to the network (host `0.0.0.0`, port from env) and connects the database.

B8. How is CORS configured?

Allowed origins come from `CORS_ORIGIN` (default includes localhost 5173 and 4173).

B9. What is the request lifecycle for a new incident?

Rate-limit check → auth check → validate body → save incident → save location → call AI service for risk → store risk → record notifications → return response.

B10. How do you keep controllers small?

Business logic lives in services (risk, notifications, geocoding, weather, nearby resources); controllers only handle HTTP in/out.

B11. What is an enrichment module?

UI features that add context around an incident/location: map, nearby facilities, weather, address. They extend the incident without changing its core data.

B12. Why not put the map inside MongoDB?

Map discovery comes from live OpenStreetMap and is ephemeral. Storing it would go stale and add no value. Only curated facilities are stored.

B13. What is the difference between a Facility and an OSM result?

Facilities are curated DB records (managed by admin). OSM results are live discovery results that are never stored.

B14. How is configuration handled?

Central `env.ts` module per service, validated at startup. Secrets come from environment only, never committed.

B15. What is the benefit of a monorepo?

One repository holds frontend, backend, AI service, and documentation, so versioning and testing stay consistent.

C. Frontend

C1. Why React 19?

Component reuse, declarative UI, and fast re-render for a responsive SOS flow. React 19 is the current stable version in our stack.

C2. Why TypeScript?

Compile-time type safety, better autocompletion, and fewer runtime bugs in a safety-critical app.

C3. Why Tailwind CSS 4?

Utility-first styling keeps the UI consistent and fast to build without maintaining separate CSS files.

C4. What is the SOS flow?

Four steps: details → location → review → done. It minimizes typing and clicks during an emergency.

C5. How does the app get location?

`requestDeviceLocation()` uses the browser Geolocation API. It returns honest states: available, denied, timeout, unavailable, unsupported.

C6. What if the user denies location permission?

The app shows the denied state and lets the user manually search an address via geocoding. It never invents coordinates.

C7. Why Leaflet instead of Google Maps?

Leaflet with OpenStreetMap tiles is free, open, and needs no API key or billing.

C8. How is the map loaded?

Lazily, via dynamic import, so the map library does not bloat the initial bundle.

C9. How does login know where to send the user?

After login, the role decides the route: ADMIN → admin dashboard, RESPONDER → responder panel, USER → dashboard.

C10. How is auth state shared across components?

Through an `AuthProvider` React context (`useAuth`) that holds the user and token.

C11. Where is the token stored?

localStorage (remember me) or sessionStorage, under the key `rakshasafe.token`.

C12. What does the dashboard show?

A welcome hero with an SOS button, summary tiles (total incidents, active, contacts, location status), emergency helplines, and the user's incident list.

C13. What are the quick-access tools?

Emergency Contacts, Nearby Resources, and Report Unsafe Area.

C14. How does the UI handle loading and errors?

With skeleton loaders, empty states, and error states with retry buttons. Aborted requests are ignored cleanly.

C15. Is the UI responsive?

Yes — it uses responsive Tailwind classes and works on mobile and desktop.

D. Backend & API**D1. What framework is the backend?**

Node.js with Express and TypeScript.

D2. Is the API REST-based?

Yes. Resource-oriented routes under `/api` with standard HTTP verbs.

D3. What is the health endpoint?

`GET /api/health` returns status plus database state (`db.state`, `db.connected`).

D4. List the main route groups.

auth, incidents, emergency-contacts, facilities, nearby-resources, nearby (OSM), weather, notifications, rescue-teams, responder, unsafe-reports, geocode, admin.

D5. How is authentication done?

JWT bearer tokens. `requireAuth` verifies the token and re-checks the user in the database.

D6. How is authorization done?

Role middleware: `requireAdmin` and `requireResponder` guard admin and responder routes.

D7. What is `requireDb``?

Middleware that returns 503 when MongoDB is not connected, so requests fail fast instead of hanging.

D8. How is a new incident created?

`POST /api/incidents`, rate-limited to 10/min per IP, validated, then persisted and scored.

D9. What does the incident controller return?

Safe incident objects — no internal fields or sensitive data leakage.

D10. How are emergency contacts managed?

`GET/POST /api/emergency-contacts` and `PATCH/DELETE /:id`. A partial-unique index enforces at

most one primary contact per user.

D11. How does the responder get only their data?

The responder routes derive identity from the JWT and filter every query by the teams the caller belongs to.

D12. What can a responder do?

List own-team assignments, view scoped incident detail, list in-app notifications, and update assignment status along the allowed workflow.

D13. What is an AdminLog?

An audit record of administrative actions: who did what, to which target, and when.

D14. How are notifications recorded?

After a successful primary write, notification documents are inserted: one IN_APP record (DELIVERED) and one per opted SMS/Email contact (honestly NOT_CONFIGURED).

D15. Why "record after success"?

So a notification is only logged when the underlying action actually happened — no false notifications.

D16. What is `recordEventSafe`?

A best-effort wrapper: a notification failure is logged but never fails or duplicates the main operation.

D17. How does the geocode route work?

It proxies OpenStreetMap Nominatim server-side, authenticated, with an 8-second timeout and country filter for India.

D18. Why proxy geocoding through the backend?

To protect the external service from abuse, keep API keys/headers server-side, and apply rate-limiting and caching.

D19. How does the nearby route work?

It queries Overpass for hospitals, clinics, police, fire stations, etc., with progressive radius search (1 km → 2.5 km → 5 km) until results are found.

D20. What error format does the API use?

Consistent JSON: `{ success: false, error: "...", details?: [...] }`.

E. Database

E1. Which database and ODM?

MongoDB with Mongoose.

E2. Why MongoDB?

Flexible document model suits varied emergency data shapes and fast iteration.

E3. How many collections?

15, each with a Mongoose schema.

E4. Name the core collections.

users, locations, incidents, incidentupdates, riskassessments, unsafeareareports, emergencycontacts, facilities, rescueteams, rescueassignments, notifications, adminlogs, disastercategories, reports, riskzones/reportcounters.

E5. What identifies a document?

MongoDB `_id` (ObjectId), validated for shape before use.

E6. How is the user schema structured?

name, email (unique), phone, passwordHash, role (USER/ADMIN/RESPONDER), isActive, language, timestamps.

E7. How do you prevent duplicate accounts?

Unique indexes on email (and phone per design), enforced by the database.

E8. How is the incident linked to its location?

By a `locationId` reference to the locations collection.

E9. How is an incident linked to a category on risk?

RiskAssessment references the incident and stores score, level, factors, model version, and assessedAt.

E10. What is a partial-unique index and where is it used?

It enforces uniqueness only for matching documents. Used so each user has at most one primary emergency contact.

E11. How does the dashboard query data?

Using MongoDB aggregation pipelines (`$group`, `$sort`, `$limit`) — all counts are live.

E12. Does the app store PII in dashboards?

No. Dashboard responses exclude stored password hashes and secret values.

F. AI / Risk Engine

F1. What is the AI service?

A Python FastAPI service exposing `POST /risk/assess` and `GET /health`.

F2. What is the model name?

`raksha-risk-v1`.

F3. What algorithm does it use?

A weighted, rule-based additive scoring algorithm — deterministic, not ML.

F4. Why not machine learning?

Explainability, reproducibility, and no training data yet. In safety systems, a transparent score is more defensible than a black box.

F5. Give the exact scoring weights.

Base by priority: LOW 15, MEDIUM 35, HIGH 60, CRITICAL 80. Disaster: +5. Verified nearby reports: +6 each, cap 30. Unverified reports: +2 each, cap 10. Active nearby incidents: +3 each, cap 15. Clamp 0–100.

F6. What are the risk bands?

Score < 25 → LOW; < 50 → MEDIUM; < 75 → HIGH; ≥ 75 → CRITICAL.

F7. What is the nearby radius for the risk engine?

2 km, using the Haversine formula.

F8. Why Haversine?

It computes great-circle distance between two lat/long points accurately for nearby ranges.

F9. What is stored with each assessment?

Score, level, contributing factors, model version, and timestamp.

F10. What is Gemini's exact role?

An optional natural-language explanation of the already-computed result. It never scores and never writes to the database.

F11. What happens if Gemini is unavailable?

The system uses a built-in deterministic explanation. The score is unaffected.

F12. How does the backend call the AI service?

Over HTTP with a configurable timeout (`AI_SERVICE_TIMEOUT_MS`, default 10000 ms).

F13. What if the AI service is down?

The backend uses its local fallback risk calculation. The SOS flow never breaks.

F14. Is the risk score stored or recomputed?

It is computed on demand and stored as a RiskAssessment, and can be re-fetched.

F15. Can two identical incidents get different scores?

No — the engine is deterministic, so identical inputs give identical scores.

F16. Where is the risk logic documented?

In `ai-service/app/risk.py`, with the weights written explicitly in code and in the documentation.

G. Security**G1. How are passwords stored?**

Hashed with bcrypt, 12 salt rounds. Plain text is never stored or returned.

G2. What is in the JWT payload?

Only `sub` (user id) and `role`. No sensitive data.

G3. How long do tokens last?

According to `JWT_EXPIRES_IN` (default 7 days).

G4. How are protected routes secured?

`requireAuth` verifies the token, and then re-loads the user from the database (preferring live DB state over the token).

G5. What is the `toSafeUser` pattern?`

A serializer that strips `passwordHash` and internal fields before sending a user to clients.

G6. How is role escalation prevented?

Public registration always forces role USER. Role changes are only possible through the admin-only update path, which is validated.

G7. How do you rate-limit abuse?

An in-memory fixed-window limiter keyed by IP and route. Incident creation and unsafe reports are capped at 10/min.

G8. What are the limiter's limitations, stated honestly?

It is per-instance (not shared across servers), so it suits a single-instance deployment.

G9. How do you prevent leaking secrets?

Secrets live only in environment variables; the repo commits `.env.example` with names, not values.

G10. How do you handle errors without leaking internals?

The global error handler returns generic messages for 500s and never exposes stack traces to clients.

G11. Is CSRF a concern?

We use bearer tokens in the Authorization header (not cookie-based sessions), which reduces classic CSRF exposure.

G12. How do responders avoid seeing others' data?

Every responder query is constrained to the caller's team memberships derived from the JWT.

H. Testing & Quality**H1. What testing levels did you use?**

Five: unit, integration, API functional, browser end-to-end, and manual.

H2. Give the unit test numbers.

36 suites, 127 checks, all passed.

H3. Give the integration test numbers.

67 cases passed.

H4. Give the API functional test numbers.

101 cases across 27 modules, all passed (live environment).

H5. Give the browser/E2E numbers.

16 journeys, 15 passed; T14 found layout defect D1.

H6. What test runner is used?

The Node.js built-in test runner with `tsx` for TypeScript.

H7. Is Supertest used?

Yes — Supertest is used in the integration tests for HTTP-level assertions, together with `mongodb-memory-server` for an in-memory database. The test runner itself is the Node.js built-in

`node:test` runner (not Jest).

H8. What linter is used?

`oxlint` (frontend), not ESLint.

H9. What defect is documented?

D1, a layout defect found by browser journey T14. It is recorded honestly in the evidence.

H10. Do you test failure cases?

Yes — including database down and AI service down, to prove graceful degradation.

H11. Why is honest testing emphasized?

Because it is a safety system; hiding defects would be unethical and dangerous.

H12. What is out of scope for testing?

Penetration testing, large-scale load testing, and CI/CD pipeline testing.

I. Deployment, Tools & Process

I1. How is the frontend deployed?

As a static build suitable for hosts like Cloudflare Pages, and optionally packaged as an

Android app with Capacitor.

I2. What is Capacitor used for?

Wrapping the web app into a native Android shell (`appId: com.rakshasafe.app`).

I3. Are deployment secrets committed?

No. Secrets are supplied as environment variables at deploy time.

I4. What tools did you use?

VS Code, Git, npm, Node.js, Python, MongoDB, and the Node test runner.

I5. What is the repository structure?

`frontend/`, `backend/`, `ai-service/`, and `documentation/` in one monorepo.

I6. What is your development port setup?

Frontend 5173, backend 5000, AI service 8000, MongoDB 27017.

I7. How is the environment kept consistent?

`.env.example` files per service document all required variables by name.

I8. How is the code organized in the backend?

config, controllers, middleware, models, routes, services, utils, validators, scripts, tests.

I9. What is the ``scripts/`` folder for?

Seed and maintenance scripts: seed admin, seed responder, update admin, purge test data, verify models, verify AI failures.

I10. Is there a README?

Yes, but it is stale — it claims no business modules are implemented. The code is the source of truth; flag this mismatch honestly if asked.

J. Future Scope & Tricky Questions

J1. What is your biggest limitation?

No real SMS/Email provider, so contact messaging is recorded as NOT_CONFIGURED rather than delivered.

J2. How would you make the AI a true ML system?

Collect real incident outcomes and train a classifier, then A/B test it against the rule engine behind the same API.

J3. How would you support real-time updates?

Add WebSockets (e.g., Socket.IO) or server-sent events to push status changes to clients.

J4. How would you make it multi-instance scalable?

Move the rate limiter to Redis and add a shared session/queue layer.

J5. How would you handle offline areas?

A service worker plus local queue that syncs incidents when connectivity returns.

J6. "Your risk engine might be wrong." How do you respond?

It is transparent and tunable — the weights are explicit and can be adjusted. For a first version, explainability and reproducibility were prioritized over opaque accuracy.

J7. "Isn't rule-based logic outdated?"

No. Many production risk/fraud engines are rule-based or hybrid because they are auditable. ML can be added later behind the same interface.

J8. How is your project different from existing apps?

It combines women safety and disaster response, uses an explainable risk engine, integrates live open map/weather data, and is honest about what it cannot do.

J9. What would you do differently if you restarted?

Add real messaging and real-time transport earlier, and add automated end-to-end coverage in CI from day one.

J10. Why should this project be considered successful?

It delivers a complete, working, tested, three-tier safety platform with real integrations, documented limitations, and honest evidence — all built and understood end to end.

J11. Can you explain your project in one sentence?

RakshaSafe turns a single SOS action into a scored, located, notified, and assigned emergency response, using an explainable AI risk engine at its core.

J12. What did you personally implement?

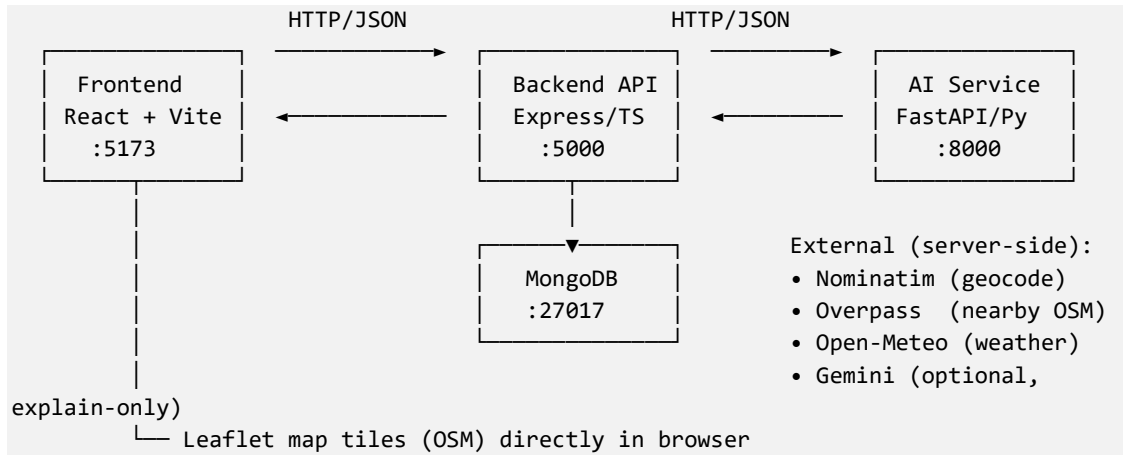
The full three-tier implementation: React frontend, Express/MongoDB backend with JWT security,

the FastAPI deterministic risk engine, external integrations, and the test suite.

05 — Technical Deep Dive

For technical examiners and follow-up questions. Everything here maps to real files.

1. High-level architecture



Key principle: **the backend owns all persistence and orchestration**. The AI service is stateless and replaceable. External discovery data is ephemeral and never stored.

2. Repository layout

```
Raksha/
├─ frontend/                # React 19 + TS + Vite + Tailwind 4 + Leaflet
│   └─ src/
│       ├── App.tsx
│       └─ pages/           # SosPage, LoginPage, RegisterPage, DashboardPage, Admin*,
│                               Responder*
│       ├── components/     # ui/*, map/SafetyMap, enrichment/LocationEnrichment,
│       ├── lib/             # api.ts, AuthProvider.tsx, auth-context, geolocation.ts,
│       └─                   # hash-route, incidents, resources, i18n, notifications,
│                               theme, useEnrichment
│       ├── vite.config.ts   # dev proxy /api -> :5000, /ai -> :8000
│       └─ capacitor.config.ts # appId com.rakshasafe.app
├─ backend/                  # Node 22 + Express + TS + Mongoose
│   └─ src/
│       ├── app.ts, server.ts
│       ├── config/          # env.ts, db.ts
│       └─ routes/           # index, auth, incidents, admin, responder, nearby, weather,
│                               # geocode, emergencyContacts, facilities, notifications,
│                               rescueTeams,
│                               # unsafeReports, nearbyResources
│       ├── controllers/     # auth, incident, risk, admin*, responder, notification, ...
│       └─ services/         # riskAssessment, geminiExplanation, notifications,
│                               geocoding,
│                               # nearbyResources, osmPlaces, weather, reports
│       ├── models/          # 15 Mongoose schemas
│       └─ middleware/       # auth.ts, requireDb.ts, rateLimit.ts, error.ts
```

```

|   | validators/    # auth.ts, incident.ts, emergencyContact.ts, nearby.ts
|   | utils/        # errors, jwt, password, geo, export, phone, search
|   | scripts/      # seedAdmin, seedResponder, updateAdmin, purgeTestData,
verify*
|   |   tests/      # unit/*.test.ts, integration/*.test.ts
| ai-service/      # Python 3.12 + FastAPI
|   | app/
|   |   main.py     # FastAPI app, GET /health, POST /risk/assess
|   |   risk.py     # raksha-risk-v1 deterministic engine
|   |   config.py
| documentation/   # baseline, test evidence, Black Book, this viva pack

```

3. API surface (backend, base `/api`)

Method & Path	Auth	Purpose
GET /health	none	Service + DB state
POST /auth/register	none	Create user (role forced USER)
POST /auth/login	none	Issue JWT
POST /auth/logout	token	Client-side token clear
GET /auth/me	token	Current safe user
PATCH /auth/profile	token	Update own profile
GET/POST /incidents	token	List own / create (rate 10/min)
GET/PATCH/DELETE /incidents/:id	token	Detail / update / cancel
GET /incidents/:id/updates	token	Status history
POST/GET /incidents/:id/risk	token	Assess / fetch risk
GET /incidents/:id/nearby-resources	token	Nearby around incident
GET/POST /emergency-contacts	token	List / create
PATCH/DELETE /emergency-contacts/:id	token	Update / delete
GET/POST /facilities	token	Curated facilities
GET /nearby-resources	token	DB + optional external
GET /nearby	token	Live OSM/Overpass discovery
GET /weather	token	Open-Meteo current weather
GET /geocode	token	Address → coordinates
GET /reverse (geocode route)	token	Coordinates → address
GET/POST /notifications	token	User notifications
GET/POST /rescue-teams	admin	Manage teams
GET/PATCH /responder/assignments	responder	Own-team assignments
GET /responder/incidents/:id	responder	Scoped incident detail
GET /responder/notifications	responder	In-app only
GET/POST/PATCH/DELETE /unsafe-reports	token	Unsafe area reports (rate 10/min)
/admin/*	admin+db	Dashboard, users, teams, facilities, reports, assignments

4. Authentication & authorization flow

11. **Register** — validate (name 2–100, valid email, valid phone, password 8–128) → hash with bcrypt (12 rounds) → save with role **USER** (forced) → return safe user.
12. **Login** — accept email or phone as identifier → find user → **bcrypt.compare** → sign JWT { sub: userId, role } → return safe user + token.
13. **Protected request** — **Authorization: Bearer <jwt>** → **requireAuth** verifies signature and expiry → loads the user from DB (live state wins over token) → attaches **req.auth**.
14. **Role gate** — **requireAdmin** / **requireResponder** check **req.auth.role**.
15. **DB gate** — **requireDb** blocks DB routes with 503 if **mongoose.connection.readyState !== 1**.

Security properties: no plain-text passwords, no secrets in tokens, safe serializers strip

passwordHash and **__v**, unique indexes prevent duplicate accounts.

5. Risk engine — `raksha-risk-v1`

Deterministic additive model. Inputs: incident priority/type, counts of verified/unverified reports, and active incidents within a 2 km radius (Haversine).

```
score = BASE[priority]                # LOW 15, MEDIUM 35, HIGH 60, CRITICAL 80
if type == DISASTER:    score += 5
score += min(verified_reports * 6, 30)
score += min(unverified_reports * 2, 10)
score += min(active_incidents * 3, 15)
score = clamp(score, 0, 100)

level = score < 25 ? LOW
       : score < 50 ? MEDIUM
       : score < 75 ? HIGH
       :             CRITICAL
```

Stored per assessment: score, level, contributing factors, **modelVersion**, **assessedAt**.

Backend fallback: if **AI_SERVICE_URL** is unreachable within **AI_SERVICE_TIMEOUT_MS**, the backend's own **riskAssessment** service computes the same logic locally. The AI service can never become a single point of failure.

Gemini layer: **geminiExplanation.ts** optionally asks Gemini for a one-paragraph explanation. It cannot change the score or write to the DB. On any failure it falls back to a

template explanation. `GEMINI_API_KEY` is optional.

6. Notification model (honest delivery)

On a successful event (`incident.created`, `incident.status.changed`, `incident.assigned`, `incident.assignment.updated`):

- 1 × **IN_APP** record → status `DELIVERED` (storing and serving *is* in-app delivery).
- 1 × **SMS** per contact with `notifyViaSms` → status `NOT_CONFIGURED` (no provider).
- 1 × **EMAIL** per contact with `notifyViaEmail` → status `NOT_CONFIGURED`.

`recordEventSafe` wraps this so a notification failure is logged but never corrupts the main operation. Responders only ever receive the **IN_APP** channel (contact-channel records reference the owner's private contacts).

7. Location honesty model

`geolocation.ts` never fabricates coordinates. It returns an explicit state:

`available` | `denied` | `timeout` | `unavailable` | `unsupported`. On failure the UI keeps the latitude/longitude (if any) and shows "unavailable", or lets the user search an address.

8. Rate limiting

`rateLimit({ windowMs, max })` — in-memory fixed-window, keyed by `ip:route`. Used on incident

creation and unsafe reports (10/min). A `reap()` routine clears expired buckets once the map exceeds 10,000 entries. Documented limitation: per-instance (use Redis for multi-instance).

9. Error handling

- Custom `HttpError` with factory helpers: `badRequest(400)`, `unauthorized(401)`, `forbidden(403)`, `notFoundError(404)`, `conflict(409)`, `serviceUnavailable(503)`.
- Global `errorHandler` maps: `HttpError` → its status; duplicate key (code 11000) → 409 with the conflicting field; Mongoose validation → 400 with field details; bad JSON → 400; payload too large → 413; unknown → 500 (generic message, full error logged server-side).
- If headers are already sent, it delegates to the default handler to avoid `ERR_HTTP_HEADERS_SENT`.

10. External integrations

Integration	Used for	Auth	Stored?
Nominatim (OSM)	Forward + reverse geocoding	none	No
Overpass (OSM)	Nearby hospitals/police/fire	none	No
Open-Meteo	Weather	none	No
Google Gemini	Optional explanation	API key (optional)	No

All external calls are server-side, time-bounded, and failure-isolated.

11. Environment variables (names only)

Backend: `NODE_ENV`, `PORT`, `MONGO_URI`, `JWT_SECRET`, `JWT_EXPIRES_IN`, `AI_SERVICE_URL`,
`AI_SERVICE_TIMEOUT_MS`, `GEMINI_API_KEY` (optional), `GEMINI_MODEL`, `GEMINI_TIMEOUT_MS`,
`CORS_ORIGIN`, `LOG_LEVEL`, `GOOGLE_PLACES_API_KEY` (optional), `NEARBY_EXTERNAL_ENABLED`,
`NEARBY_EXTERNAL_TIMEOUT_MS`, `NEARBY_RADIUS_KM`.

AI service: `SERVICE_NAME`, `VERSION`, `HOST`, `PORT`, `LOG_LEVEL`, `CORS_ORIGINS`.

Frontend: `VITE_API_BASE_URL`, `VITE_PROXY_TARGET`, `VITE_AI_PROXY_TARGET`.

Never read out secret values. Refer to variables by name only.

12. Testing strategy

Five levels, with these recorded results:

- **Unit** — 36 suites, 127 checks, all passed.
- **Integration** — 67 cases passed.
- **API functional (live)** — 101 cases across 27 modules, all passed.
- **Browser / E2E** — 16 journeys, 15 passed; T14 found layout defect D1.
- **Manual** — exploratory and accessibility checks.

Run with the Node built-in test runner (`node:test`) via `tsx`. Integration tests use

Supertest for HTTP assertions and **mongodb-memory-server** for an in-memory database.

Linting uses `oxlint`. Jest is not used.

13. Data model (15 collections, summaries)

Collection	Purpose	Notable fields
users	Accounts	name, email(unique), phone, passwordHash, role, isActive
locations	Captured coordinates	latitude, longitude, address, accuracy, capturedAt

incidents	Emergency reports	type, category, priority, status, userId, locationId
incidentupdates	Status history	statusFrom, statusTo, comment, updatedBy
riskassessments	Risk results	riskScore, riskLevel, factors, modelVersion, assessedAt
unsafeareareports	Community reports	category, severity, isVerified, coordinates
emergencycontacts	Per-user contacts	name, phone, email, notifyViaSms/Email, isPrimary
facilities	Curated resources	facilityType, isOperational, coordinates, phone
rescueteams	Field teams	name, members[], isActive
rescueassignments	Incident↔team	incidentId, teamId, status
notifications	Event records	incidentId, channel, status, attemptCount
adminlogs	Audit trail	adminId, action, targetType, targetId, ipAddress
disastercategories	Disaster taxonomy	code, name
reports	Generated reports	type, range, counts
riskzones / reportcounters	Aggregation helpers	zone data / sequence counters

14. Deployment shape

- Frontend: static build (Cloudflare Pages-style) or Android via Capacitor.
- Backend: Node process bound to **0.0.0.0** on **PORT**.
- AI service: Uvicorn/FastAPI on **PORT**.
- Database: MongoDB connection via **MONGO_URI** with bounded exponential retry.
- Secrets: environment only; never committed.

15. Design decisions worth defending

- **Deterministic AI over ML** — explainability and reproducibility first.
- **Fallback everywhere** — AI down, DB down, external APIs down: the core SOS flow survives.
- **Honest states** — never fake a location or a delivered message.
- **Safe serializers** — sensitive fields stripped at the boundary.
- **Separation of services** — independent change and scaling.
- **Live aggregations for the dashboard** — no hard-coded metrics.

06 — Live Demo Checklist

*Follow top to bottom. Do the **pre-flight** before the viva starts. Have the fallback plan ready in case Wi-Fi or the database misbehaves.*

A. Pre-flight (do 15–20 minutes before)

16. **MongoDB running** — local service on 27017 (`mongodb://127.0.0.1:27017/rakshasafe`).

- Quick check: open a Mongo shell or Compass and confirm the server accepts connections.

17. **Env files present** — `backend/.env` copied from `.env.example`, `frontend/.env` copied from `.env.example`, `ai-service` env if needed. Never show secret values on screen.

18. **Dependencies installed** — run once, ahead of time:

- `backend\:` `npm install`
- `frontend\:` `npm install`
- `ai-service\:` create venv and `pip install -r requirements.txt`

19. **Seed accounts** (so you don't rely on registering live):

- `backend\:` `npm run seed:admin`
- `backend\:` `npm run seed:responder`
- Keep the admin + responder credentials handy (do NOT display passwords on the projector).

20. **Build sanity** (optional but good): `backend\:` `npm run typecheck`, `frontend\:` `npm run build`.

21. **Browser** — open two tabs (or two browsers): one for the USER, one for the ADMIN. Keep a third ready for the RESPONDER if time allows.

22. **Screen** — close email/chat; set browser zoom ~100%; have this checklist on a second device, not the projected screen.

B. Start the three services (3 terminals)

Open three terminals in the repo root and run:

```
# Terminal 1 — AI service (start first; core SOS does not depend on it)
cd ai-service
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

# Terminal 2 — Backend API
cd backend
npm run dev

# Terminal 3 — Frontend
```

```
cd frontend
npm run dev
```

Expected:

- AI service: Uvicorn running on <http://0.0.0.0:8000>.
- Backend: listening on port [5000](#); DB connect log; do not paste the MONGO_URI into slides.
- Frontend: Vite ready on <http://localhost:5173>.

Health check first (always demo this):

```
curl http://localhost:5000/api/health
```

Expect JSON with a healthy status and `db.connected: true`. If DB is false, fix the DB before continuing (see Fallback).

C. Demo script (10–12 minutes)

Step 1 — Landing & Login (1 min)

23. Open <http://localhost:5173>.

24. Point out the landing hero, the four feature cards (Women Safety, Disaster Response, Live Location, Rescue Teams), and the navbar info dialogs.

25. Log in as the USER. Note the role-based redirect to [#/dashboard](#).

Step 2 — Dashboard (1 min)

26. Show the welcome hero, SOS button, and summary tiles (total incidents, active, contacts, location status).

27. Show the Emergency Helplines section (static national numbers — always available even offline).

28. Note the quick-access tiles: Contacts, Nearby Resources, Report Unsafe Area.

Step 3 — Raise an incident (SOS) (3 min) — the core demo

29. Click **SOS**. Walk the four steps: **details** → **location** → **review** → **done**.

30. Show that the browser asks for GPS permission.

- Allow it → location captured and shown on the review step.

31. Fill a realistic scenario, e.g. category "Women Safety", priority HIGH, a clear description.

32. On **review**, show the summary, then **submit**.

33. After submit:

- Show the incident appear on the dashboard with its status badge.

- Open the incident detail and show the **risk assessment** (score + level + factors).
- Show the map with the pinned location and nearby facilities.

Step 4 — Risk assessment (1–2 min)

34. Open **POST/GET risk** result in the UI.
35. Explain the score is **raksha-risk-v1**, deterministic, with documented weights.
36. (Optional, if Gemini configured) show the explanation paragraph; state it explains but never changes the score.

Step 5 — Nearby & enrichment (1 min)

37. Show nearby hospitals/police from OpenStreetMap (Overpass) on the map.
38. Show the weather tile (Open-Meteo).
39. State clearly: OSM discovery results are never stored.

Step 6 — Admin view (2–3 min)

40. Switch to the ADMIN tab; log in.
41. Open the **Admin Dashboard**: user count, incidents by status/priority/type, recent incidents, teams, assignments, facilities, report verification, risk distribution, recent activity — all live aggregations.
42. Show **assign an incident to a rescue team** (status moves to ASSIGNED).
43. Show the audit log entry created by the action.

Step 7 — Responder view (1–2 min, if time)

44. Log in as RESPONDER in the third tab.
45. Show their panel lists **only** their team's assignments.
46. Open a scoped incident detail.
47. Move the assignment **ASSIGNED → EN_ROUTE → ON_SCENE → COMPLETED**.
48. Return to admin/user to show the status reflected.

Step 8 — Honest engineering points (30 sec)

- In-app notification recorded as DELIVERED.
- SMS/Email records honestly show NOT_CONFIGURED (no provider configured).
- Location denial path: mention it shows "denied" and offers manual address search.

D. What to say if a service is down

Failure	Fallback line to say
AI service down	"The backend detects the AI service is unreachable and uses its deterministic local fallback — the SOS flow still works."
MongoDB down	"DB-dependent routes return a clean 503 via requireDb ; enrichment routes like weather and nearby keep working."

Internet/OSM/weather down	"External discovery is optional; the app shows an unavailable state and core flow continues."
Frontend build issue	Fall back to the committed screenshots in documentation/test-evidence/screenshots/ .
No GPS on the machine	Use the manual address search step; say "we never fabricate coordinates."

E. Evidence to have open (just in case)

- [documentation/test-evidence/README.md](#) and the screenshots folder.
- Chapter 6 test summary in the Black Book.
- This checklist on a second device.

F. Do's and Don'ts

Do

- Start with [/api/health](#).
- Demo the SOS flow as your headline.
- Speak the honest limitations proactively.

Don't

- Never show [.env](#) contents or any key/password.
- Don't claim SMS/Email is delivered.
- Don't quote test numbers other than the recorded ones.
- Don't navigate into [node_modules](#) or private caches on the projected screen.

G. Rapid command reference

```
# Backend
cd backend
npm run dev           # start API
npm run typecheck     # types
npm test              # unit + integration
npm run verify:models # model schema sanity
npm run verify:ai-failures

# Frontend
cd frontend
npm run dev           # start UI
npm run build         # production build
npm run preview       # preview build

# AI service
cd ai-service
python -m uvicorn app.main:app --port 8000 --reload
```

07 — Black Book Revision (Chapter by Chapter)

Aligned to the actual Black Book. Use this to connect your spoken answers to the printed report. Where the report and the code differ, be honest and say so (see the last section).

Document identity (know these by heart)

Field	Value
Title	RakshaSafe — AI-Powered Women Safety and Disaster Emergency Response System
Student	Vivek Lorikprasad Sahani
Roll No.	42
Guide	Asst. Prof. Sourabh Suryawanshi
College	D.T.S.S. College of Commerce (Autonomous), Mumbai – 400097
Academic Year	2026–27

Chapter 1 — Introduction

Remember these points:

- Background of women safety and disaster management in India.
- Problem statement: delay and poor coordination during emergencies.
- Objectives of the system (fast reporting, automatic risk scoring, nearby help, coordinated response).
- Scope: web app covering safety + disaster incidents, with roles USER / ADMIN / RESPONDER.
- Organization of the report.

Likely question: *"What was your problem statement?"* → Delay in reporting and lack of a single location-aware, risk-scored platform connecting citizens to responders.

Chapter 2 — Survey of Technology

Remember:

- Why React 19 + TypeScript + Vite + Tailwind 4 for the frontend.
- Why Node.js + Express + TypeScript + Mongoose for the backend.
- Why MongoDB for the database.
- Why Python + FastAPI for the AI/risk service.
- Why Leaflet + OpenStreetMap instead of paid map SDKs.
- Comparison with alternatives (e.g., Angular/Vue, Django/Rails, PostgreSQL/MySQL, Google Maps).

Likely question: *"Why MongoDB over a relational database?"* → Flexible document model for

varied emergency data, fast iteration, and natural nesting of location and context data.

*Note: Do **not** modify Chapter 2 of the final report. Revise from it only.*

Chapter 3 — Requirement and Analysis

Remember:

- Functional requirements: authentication, incident CRUD, risk assessment, nearby resources, emergency contacts, notifications, unsafe reports, rescue teams, admin management.
- Non-functional requirements: security, availability, explainability, graceful degradation, usability, honesty (no fabricated data).
- Feasibility: technical, economic, operational.
- Requirements baseline (see [documentation/REQUIREMENTS_BASELINE.md](#)).
- Use-case analysis: citizen raises incident; admin manages/assigns; responder updates.
- Incident categories: Women Safety, Medical Emergency, Accident, Fire, Flood, Earthquake,

Other Emergency.

Likely question: "What are your functional vs non-functional requirements?" → Answer using the two lists above; emphasize graceful degradation and explainability as non-functional requirements.

*Note: Do **not** modify Chapter 3 of the final report. Revise from it only.*

Chapter 4 — System Design

Remember:

- Three-tier architecture diagram (frontend / backend / AI service / MongoDB).
- Module design and interactions.
- DFD levels, use-case diagram, ER diagram, class/sequence design as shown.
- Database schema design for the collections.
- API design (REST under [/api](#)).
- The risk engine design: [raksha-risk-v1](#) deterministic weights and bands.

Likely question: "Draw/explain your architecture." → Three tiers, backend orchestrates, AI service stateless with fallback, MongoDB persistence, external services server-side.

Likely question: "Explain your ER/collections." → Refer to the 15 collections summary in

[05-Technical-Deep-Dive.md](#).

Chapter 5 — System Implementation

Remember:

- Frontend implementation: components, pages (SOS, Dashboard, Admin, Responder), hash routing, map, location honesty, i18n, theme.
- Backend implementation: routes → controllers → services → models, middleware (auth, requireDb, rateLimit, error), validators, utils.
- AI service implementation: FastAPI app, `POST /risk/assess`, `risk.py` engine, optional Gemini explanation.
- Integrations: Nominatim, Overpass, Open-Meteo, Gemini.
- Security implementation: bcrypt, JWT, safe serializers, role guards.
- Notification implementation: in-app DELIVERED; SMS/Email NOT_CONFIGURED.

Likely question: *"Show where the risk scoring is implemented."* → `ai-service/app/risk.py` (engine) and `backend/src/services/riskAssessment.ts` (fallback + orchestration).

Chapter 6 — System Testing

Remember the exact results:

- **Unit:** 36 suites, 127 checks, all passed.
- **Integration:** 67 cases passed (Supertest + mongodb-memory-server).
- **API functional (live):** 101 cases across 27 modules, all passed.
- **Browser / E2E:** 16 journeys, 15 passed; T14 found layout defect **D1**.
- **Levels:** 5 (unit, integration, API functional, browser E2E, manual).
- Out of scope: penetration testing, large-scale load testing, CI/CD testing.

Likely question: *"What defect did you find and how did you handle it?"* → T14 found layout defect D1; documented honestly in the evidence rather than hidden.

Likely question: *"Show a test case."* → Pick one unit test (e.g., risk bands) and one integration test (e.g., register → login → create incident).

Chapter 7 — Conclusion and Future Scope

Remember:

- Conclusion: a complete, tested, three-tier safety platform with explainable AI and honest

engineering.

- Limitations: no real SMS/Email provider; rule-based (not ML) risk; single-instance rate limiter; dependence on external map/weather availability.
- Future scope: real SMS/Email providers, real-time WebSockets, ML risk model once data exists, offline support, more Indian languages, wearables/shake-to-SOS, mobile app via Capacitor.

Likely question: "What is your future scope?" → List the items above in priority order.

Chapter 8 — Appendix / Supporting Material

- SRS tables and detailed test-evidence tables.
- Screenshots in [documentation/test-evidence/screenshots/](#).
- Diagrams in [chapter6_diagrams_bw/](#).

Black Book vs actual code — honest mismatches (be ready)

#	Report / older doc says	Actual implementation	How to answer
1	README.md claims "no business modules implemented yet"	System is fully implemented	"The README is stale; it predates the build. The code and Chapter 6 evidence are current."
2	Scope baseline lists USER and ADMIN	Code adds a third role RESPONDER	"RESPONDER was added as a deliberate extension for field rescue teams, documented in the design."
3	Testing tools mentioned as Jest/Supertest in places	Node node:test runner + tsx ; Supertest used for integration; no Jest	"We use the Node built-in test runner. Supertest is used at the HTTP integration level. Jest is not used."
4	Linting implied as ESLint	Frontend uses oxlint	"We use oxlint for fast linting."
5	Messaging imagined as delivered	SMS/Email honestly NOT_CONFIGURED ; only in-app is DELIVERED	"Delivery for SMS/Email is intentionally marked NOT_CONFIGURED because no provider is configured."
6	AI sometimes described loosely	Risk engine is deterministic rule-based; Gemini is explain-only	"The scoring is deterministic; Gemini only explains, it never scores."

If an examiner finds one of these, agree calmly, state the accurate version, and show the

code or the test evidence. Honesty here is a strength.

60-second chapter sweep (memorize)

- 49. Intro — problem, objectives, scope.
- 50. Survey — why this stack.
- 51. Requirements — functional, non-functional, use cases.
- 52. Design — 3-tier architecture, 15 collections, REST API, risk model.
- 53. Implementation — routes/controllers/services/models, auth, integrations, notifications.
- 54. Testing — 36/127, 67, 101/27, 16 journeys, defect D1, 5 levels.
- 55. Conclusion — honest limitations + future scope.
- 56. Appendix — evidence and diagrams.

08 — Last-Minute Revision

Read this in the final 30–60 minutes. Numbers, names, and quick answers only.

The 15 numbers/names to memorize

Item	Value
Project name	RakshaSafe
Full title	AI-Powered Women Safety and Disaster Emergency Response System
Student / Roll	Vivek Lorikprasad Sahani / 42
Guide	Asst. Prof. Sourabh Suryawanshi
Frontend port	5173
Backend port	5000
AI service port	8000
Risk model name	raksha-risk-v1
Risk bands	<25 LOW, <50 MEDIUM, <75 HIGH, ≥75 CRITICAL
Base scores	LOW 15, MEDIUM 35, HIGH 60, CRITICAL 80
Risk nearby radius	2 km
Collections in MongoDB	15
Unit tests	36 suites / 127 checks, all passed
Integration tests	67 cases passed
API functional tests	101 cases / 27 modules, all passed
Browser E2E	16 journeys, 15 passed, defect D1

The one-line answers (rapid fire)

- **3 tiers:** React frontend → Express backend → FastAPI AI service, with MongoDB.
- **AI is:** deterministic weighted risk scoring; Gemini only explains.
- **Auth:** JWT + bcrypt (12 rounds) + role middleware.
- **Roles:** USER, ADMIN, RESPONDER.
- **Workflow:** REPORTED → ACKNOWLEDGED → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED (+CANCELLED).
- **Notifications:** in-app DELIVERED; SMS/Email NOT_CONFIGURED.
- **Location:** never fabricated; honest states.
- **Maps:** Leaflet + OpenStreetMap; weather via Open-Meteo; geocode via Nominatim.
- **Testing runner:** Node **node:test** + tsx; Supertest for integration; oxlint for lint.
- **Failures handled:** AI down → local fallback; DB down → clean 503; external down → unavailable state.

The exact risk formula (say it in one breath)

Base by priority (15/35/60/80), +5 if disaster, +6 per verified report (cap 30), +2 per

unverified report (cap 10), +3 per active nearby incident (cap 15), clamp 0–100, then band it.

The 8 honest points (say these before they ask)

57. Risk engine is **rule-based, not ML** — for explainability.

- 58. Gemini is **optional and explain-only**.
- 59. SMS/Email are **NOT_CONFIGURED** — no provider.
- 60. Rate limiter is **in-memory** (single-instance).
- 61. **README.md** is **stale** — code is current.
- 62. RESPONDER is an **added** role beyond the original scope.
- 63. Test runner is **node:test**, not Jest.
- 64. Defect **D1** was found by E2E journey T14 and documented.

Revision order (if you only have 30 minutes)

- 65. This file (8 min) — numbers and one-liners.
- 66. **01-Project-Overview.md** elevator pitch (4 min).
- 67. **04** groups **F (AI)**, **G (Security)**, **H (Testing)** (10 min).
- 68. **06** demo script sections B–C (5 min).
- 69. **07** mismatches table (3 min).

If you have 2 hours

- 70. Read **02** (easy answers) fully.
- 71. Read **03** (Hinglish) fully and speak a few aloud.
- 72. Read **05** (technical deep dive) fully.
- 73. Skim **04** all groups.
- 74. Rehearse the demo once end to end.

Do's and Don'ts (final)

Do

- Breathe, then answer in 3–6 sentences.
- Start technical answers from the architecture, then drill down.
- Admit limitations confidently — it shows maturity.
- Quote only recorded test numbers.
- Say "I'll check the code" rather than guessing.

Don't

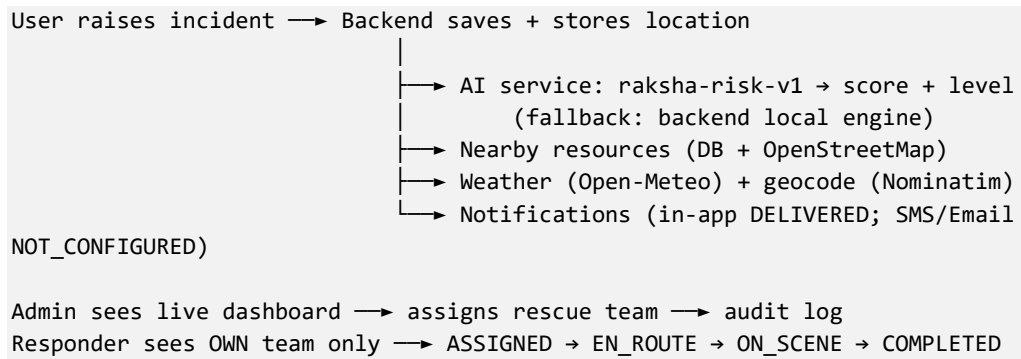
- Don't memorize exact query syntax—explain the approach.
- Don't claim ML/AI capabilities the engine doesn't have.
- Don't say "sent SMS/Email"; say "recorded as not configured."
- Don't show **.env** or any keys/passwords.
- Don't read the stale README as fact.

Emergency phrases

- "The scoring is deterministic, so identical inputs give identical scores."
- "If the AI service is down, the backend uses its local fallback — the SOS flow never breaks."
- "Passwords are bcrypt-hashed; plain text is never stored."

- "We scope every responder query to their own teams."
- "This is documented as a known limitation in Chapter 7."

Mental model of the whole system (one picture)



Good luck. You built this — speak like the owner.